Fangyuan Shi

Atlanta, GA | (470) 343-3927 | fshi46@gatech.edu | linkedin.com/in/fangyuan-shi | github.com/Farryn1

Summary

M.S. Operations Research candidate at Georgia Tech (GPA 3.87) with a B.Eng. in Industrial Engineering from Sichuan University (GPA 3.89). Specializes in mathematical optimization, applied machine learning, and time-series forecasting. Hands-on experience implementing large-scale MILP decomposition (Benders decomposition, Lagrangian relaxation), data-driven PDE modeling, LLM-based black-box optimization (OPRO), and multi-model energy price forecasting benchmarks.

Education

Georgia Institute of Technology — Atlanta, GA
M.S., Operations Research — GPA: 3.87/4.0 — *Aug. 2025 – Present*

Sichuan University — Chengdu, China
B.Eng., Industrial Engineering — GPA: 3.89/4.0 — *Sep. 2021 – Jun. 2025*

Standardized Tests: GRE 328 | TOEFL 101
Relevant Coursework: Deterministic Optimization, Integer Programming, Stochastic Processes, Machine Learning, Supply Chain Engineering, Data Analytics

Projects

Wildfire Spread Modeling via Reaction-Diffusion PDE | *Python, NumPy, SciPy, Optuna* | GitHub Aug. 2025 – Dec. 2025

- Fit a spatially-heterogeneous reaction-diffusion PDE (temperature field + fuel consumption, Arrhenius kinetics) to 909-frame infrared satellite video covering ∼9 hours of real wildfire spread.
- Calibrated region-specific parameters (diffusivity, decay rate, initial fuel) for fire vs. background pixels separately via Bayesian optimization with Optuna (TPE sampler, 30 trials) using a fire-weighted MSE loss ($w_{\text{fire}} = 20$) to handle class imbalance.
- Implemented CFL-adaptive explicit Euler integration with Neumann boundary conditions; conducted stability analysis over four time-step safety factors; PDE model outperformed persistence baseline on fire-region MSE over the full 909-frame rollout.
- Extracted per-frame temperature fields from raw infrared video via colorbar LUT inversion (nearest-neighbor RGB lookup); normalized to [0,1] using ambient (32.61°C) and peak-fire (95°C) reference temperatures, masking colorbar and timestamp overlay regions from loss computation.

Facility Location: MILP, Benders & Lagrangian Relaxation | *Python, PuLP, SciPy* | GitHub Jan. 2026 – May 2026

- Formulated the Capacitated Facility Location Problem (CFLP) and solved it three ways: direct MILP via PuLP/CBC, Benders decomposition (master MILP + LP subproblem duals generating optimality and feasibility cuts), and Lagrangian relaxation (per-facility subproblems solved in closed form, subgradient updates with Polyak step size).
- Benders decomposition converged to the MILP-optimal solution on all instance sizes; Lagrangian relaxation delivered near-optimal feasible solutions significantly faster, demonstrating the accuracy–speed trade-off central to large-scale optimization practice.
- Benchmarked across small/medium/large synthetic instances using only open-source solvers (CBC, HiGHS); produced convergence curves, runtime-scaling analysis, and optimality-gap comparison.
- Maintained per-iteration lower bound (master objective) and upper bound (best feasible solution) trajectories for both decomposition methods, providing rigorous stopping criteria and visualizing the rate at which each method closes the optimality gap.

LLM as Optimizer (OPRO): Black-Box & Combinatorial Search | *Python, Claude API* | GitHub Jan. 2026 – May 2026

- Implemented the OPRO framework (Yang et al., 2023) in which an LLM iteratively proposes candidate solutions from a natural-language meta-prompt encoding the top-k evaluated pairs sorted worst-to-best, enabling the model to "learn" the optimization landscape from its own trajectory.

- Evaluated on multimodal continuous benchmarks (Rastrigin 2-D/10-D, Ackley 5-D) and a Traveling Salesman Problem; compared against random search, hill-climbing, and 2-opt local search under equal evaluation budgets.
- Integrated Claude API as the real LLM backend with a deterministic mock fallback for offline reproducibility; OPRO showed monotonically decreasing best-so-far curves, competitive with classical baselines under matched budgets.
- Designed a robust output parser with regex extraction and random-candidate fallback on parse failure to ensure the loop never stalls; analyzed sensitivity to trajectory window size (top-k) and iteration budget.

Henry Hub Natural Gas Forecasting | *Python, statsmodels, arch, XGBoost, PyTorch* | GitHub Aug. 2025 – Dec. 2025

- Benchmarked nine model families on Henry Hub spot prices (daily 1997–2026, 7,348 obs.; weekly with EIA/FRED/NOAA exogenous regressors): ARIMA, SARIMA, ARIMAX, SARIMAX, ARMA–GARCH, XGBoost, LSTM (PyTorch), Silverkite (Greykite), and naïve baselines.
- Rolling ARMA(3,0,3)–GARCH(1,1) with Student-t innovations achieved MAPE 3.36% on daily data (vs. naïve 3.49%); XGBoost with a 23-dimensional feature matrix (price/return lags, storage, WTI, HDD, cyclic calendar) achieved MAPE 4.70% weekly — best across all weekly models.
- Demonstrated that rolling 1-step-ahead refit reduces MAPE $\sim$4$\times$ vs. fixed multi-step forecasting across all model families; identified LSTM underperformance as a sample-size issue ($\sim$800 training observations after inner join).
- Modeled return-level volatility via two-stage ARMA–GARCH: ARMA mean equation fit first, GARCH(1,1) with Student-t innovations applied to residuals; forecast price levels recovered by exponentiating ARMA mean plus last log-price, with GARCH $\hat{\sigma}_{t+1|t}$ providing prediction interval widths.

Technical Skills

Programming: Python (NumPy, pandas, scikit-learn, PyTorch, statsmodels, arch, PuLP, Optuna, XGBoost, Greykite), R, MATLAB
Optimization: MILP modeling, Benders decomposition, Lagrangian relaxation, subgradient methods, convex optimization, PDE-constrained calibration
Machine Learning & Statistics: XGBoost, LSTM, ARMA–GARCH, SARIMA/SARIMAX, time-series forecasting, Bayesian optimization, LLM prompting
Tools: Git, Jupyter, LaTeX, HiGHS, CBC (COIN-OR)